COMPLETE CRM DATABASE

MongoDB Database Design & Architecture

A practical, scalable foundation for a multi-tenant Customer Relationship Management system.

Technology	Design
Database	MongoDB
Recommended ODM	Mongoose (Node.js)
Architecture	Multi-tenant SaaS
Coverage	Sales, leads, customers, support, billing, users, reporting

Prepared as an implementation-oriented reference document.

Contents

1. Architecture overview
2. Collection inventory
3. Core collection document structures
4. Additional production collections
5. Relationships and data flow
6. Index strategy and mongosh script
7. Validation, security and production rules
8. Suggested backend project structure
9. Implementation checklist

1. Architecture Overview

Database name: **crm_database**. The design supports a single CRM installation and can grow into a multi-tenant SaaS platform where multiple organizations use the same application while their data remains logically isolated.

Core design principles:

• Use references for independently managed entities such as users, customers, leads, deals and invoices.

• Use embedded documents for bounded data that is normally read together, such as invoice line items and postal addresses.

• Include organizationId in every organization-owned document and enforce tenant scoping in every backend query.

• Keep historical events such as status changes and audit activity in separate collections.

• Use MongoDB schema validation plus application-level validation.

2. Collection Inventory

Collection	Purpose
organizations	Registered businesses / tenants
users	CRM users and account profiles
roles	Role definitions and permissions
teams	Sales and support teams
customers	Customer accounts / companies
contacts	People associated with customers
leads	Potential customers
lead_sources	Website, referral, social media, campaigns
pipelines	Sales pipelines
pipeline_stages	Stages within a pipeline
deals	Sales opportunities
products	Products and services
activities	Calls, emails, meetings and interactions
tasks	Assigned work and follow-ups
notes	Notes attached to CRM records
tickets	Customer support cases
ticket_messages	Ticket conversations
quotes	Customer quotations
invoices	Invoices and line items
payments	Payment transactions
notifications	In-app notifications
files	File metadata and storage paths
tags	Reusable labels
custom_fields	Organization-defined fields

Collection	Purpose
audit_logs	Record of user/system actions
settings	Organization-level preferences
counters	Atomic sequence counters
lead_status_history	Lead status history
deal_stage_history	Deal stage movement history
email_templates	Reusable email content
email_campaigns	Marketing campaigns
email_logs	Email delivery history
sequences	Automated follow-up sequences
reminders	Scheduled reminders
integrations	Third-party integration configuration
webhooks	Outgoing event webhook configuration

3. Core Collection Document Structures

The examples below are MongoDB shell-style document templates. ObjectId() and ISODate() are illustrative constructors; generate real values in the application. Store passwords only as secure password hashes.

organizations

```
{
  _id: ObjectId(),
  name: "ABC Software House",
  slug: "abc-software-house",
  email: "info@example.com",
  phone: "+923001234567",
  website: "https://example.com",
  address: {
    country: "Pakistan", state: "Sindh",
    city: "Karachi", street: "Main Shahrah-e-Faisal",
    postalCode: "75000"
  },
  logo: null,
  timezone: "Asia/Karachi",
  currency: "PKR",
  subscription: {
    plan: "professional",
    status: "active",
    expiresAt: ISODate()
  },
  status: "active",
  createdAt: ISODate(),
  updatedAt: ISODate()
}
```

users

```
{
  _id: ObjectId(),
  organizationId: ObjectId(),
  firstName: "Muhammad",
  lastName: "Ali",
  email: "ali@example.com",
  password: "SECURE_HASH_ONLY",
  phone: "+923001234567",
  avatar: null,
  roleId: ObjectId(),
  teamId: ObjectId(),
  designation: "Sales Manager",
  status: "active",
  isEmailVerified: true,
  lastLoginAt: ISODate(),
  createdBy: ObjectId(),
```

```
  deletedAt: null,
  createdAt: ISODate(),
  updatedAt: ISODate()
}
```

roles

```
{
  _id: ObjectId(),
  organizationId: ObjectId(),
  name: "Sales Manager",
  slug: "sales-manager",
  permissions: [
    "leads.view", "leads.create", "leads.edit",
    "deals.view", "deals.create", "deals.edit",
    "customers.view", "customers.create",
    "reports.view"
  ],
  isSystem: false,
  createdAt: ISODate(),
  updatedAt: ISODate()
}
```

teams

```
{
  _id: ObjectId(),
  organizationId: ObjectId(),
  name: "Sales Team",
  description: "Sales department",
  managerId: ObjectId(),
  members: [ObjectId(), ObjectId()],
  status: "active",
  createdAt: ISODate(),
  updatedAt: ISODate()
}
```

customers

```
{
  _id: ObjectId(),
  organizationId: ObjectId(),
  customerCode: "CUS-0001",
  type: "company",
  name: "ABC Technologies",
  email: "info@abc.com",
  phone: "+923001234567",
  website: "https://abc.com",
  industry: "Information Technology",
  companySize: "50-100",
  taxNumber: null,
  address: {
    country: "Pakistan", city: "Karachi",
    street: "Clifton", postalCode: "75600"
  },
  assignedTo: ObjectId(),
  status: "active",
  tags: ["vip", "enterprise"],
  createdBy: ObjectId(),
  deletedAt: null,
  createdAt: ISODate(),
  updatedAt: ISODate()
}
```

contacts

```
{
  _id: ObjectId(),
  organizationId: ObjectId(),
  customerId: ObjectId(),
  firstName: "Ahmed",
  lastName: "Khan",
  email: "ahmed@abc.com",
  phone: "+923001234567",
  alternatePhone: null,
  designation: "CEO",
  department: "Management",
  isPrimary: true,
  socialProfiles: {
    linkedin: null, facebook: null, twitter: null
  },
  assignedTo: ObjectId(),
  createdAt: ISODate(),
  updatedAt: ISODate()
}
```

leads

```
{
  _id: ObjectId(),
  organizationId: ObjectId(),
  leadCode: "LEAD-0001",
  firstName: "Usman",
  lastName: "Ahmed",
  companyName: "XYZ Solutions",
  email: "usman@example.com",
  phone: "+923001234567",
  sourceId: ObjectId(),
  status: "new",
  rating: "hot",
  score: 85,
  industry: "Software",
  budget: NumberDecimal("500000.00"),
  currency: "PKR",
  assignedTo: ObjectId(),
  teamId: ObjectId(),
  address: { country: "Pakistan", city: "Karachi" },
  description: "Interested in CRM software",
  converted: {
    isConverted: false,
    customerId: null,
    contactId: null,
    dealId: null,
    convertedAt: null
  },
  tags: ["software", "high-priority"],
  customFields: {},
  createdBy: ObjectId(),
  deletedAt: null,
  createdAt: ISODate(),
  updatedAt: ISODate()
}
```

lead_sources

```
{
  _id: ObjectId(),
  organizationId: ObjectId(),
  name: "Website",
  slug: "website",
  description: "Leads generated from website",
  status: "active",
  createdAt: ISODate(),
  updatedAt: ISODate()
}
```

pipelines

```
{
  _id: ObjectId(),
  organizationId: ObjectId(),
  name: "Sales Pipeline",
  description: "Default sales pipeline",
  isDefault: true,
  status: "active",
  createdAt: ISODate(),
  updatedAt: ISODate()
}
```

pipeline_stages

```
{
  _id: ObjectId(),
  organizationId: ObjectId(),
  pipelineId: ObjectId(),
  name: "Negotiation",
  slug: "negotiation",
  order: 3,
  probability: 60,
  type: "open", // open, won, lost
  createdAt: ISODate(),
  updatedAt: ISODate()
}
```

deals

```
{
  _id: ObjectId(),
  organizationId: ObjectId(),
  dealCode: "DEAL-0001",
  title: "CRM Software Development",
```

```
  description: "Custom CRM development project",
  customerId: ObjectId(),
  contactId: ObjectId(),
  leadId: ObjectId(),
  pipelineId: ObjectId(),
  stageId: ObjectId(),
  amount: NumberDecimal("500000.00"),
  currency: "PKR",
  probability: 60,
  expectedClosingDate: ISODate(),
  actualClosingDate: null,
  status: "open", // open, won, lost
  lostReason: null,
  assignedTo: ObjectId(),
  createdBy: ObjectId(),
  createdAt: ISODate(),
  updatedAt: ISODate()
}
```

products

```
{
  _id: ObjectId(),
  organizationId: ObjectId(),
  name: "CRM Enterprise",
  sku: "CRM-001",
  description: "Enterprise CRM solution",
  type: "service",
  category: "Software",
  price: NumberDecimal("100000.00"),
  currency: "PKR",
  taxRate: 18,
  status: "active",
  createdAt: ISODate(),
  updatedAt: ISODate()
}
```

activities

```
{
  _id: ObjectId(),
  organizationId: ObjectId(),
  type: "meeting", // call, email, meeting, sms, whatsapp
  subject: "Client Requirement Discussion",
  description: "Discussed project requirements",
  relatedTo: { type: "lead", id: ObjectId() },
  assignedTo: ObjectId(),
  scheduledAt: ISODate(),
  completedAt: null,
  duration: 30,
  outcome: null,
  status: "scheduled",
  createdBy: ObjectId(),
  createdAt: ISODate(),
  updatedAt: ISODate()
}
```

tasks

```
{
  _id: ObjectId(),
  organizationId: ObjectId(),
  title: "Follow up with client",
  description: "Discuss pricing and requirements",
  relatedTo: { type: "lead", id: ObjectId() },
  assignedTo: ObjectId(),
  createdBy: ObjectId(),
  priority: "high", // low, medium, high, urgent
  status: "pending", // pending, in_progress, completed, cancelled
  dueDate: ISODate(),
  completedAt: null,
  reminderAt: ISODate(),
  createdAt: ISODate(),
  updatedAt: ISODate()
}
```

notes

```
{
  _id: ObjectId(),
  organizationId: ObjectId(),
  title: "Client Discussion",
  content: "Client wants enterprise package.",
  relatedTo: { type: "customer", id: ObjectId() },
  createdBy: ObjectId(),
  isPinned: false,
```

```
  createdAt: ISODate(),
  updatedAt: ISODate()
}
```

tickets

```
{
  _id: ObjectId(),
  organizationId: ObjectId(),
  ticketNumber: "TKT-0001",
  customerId: ObjectId(),
  contactId: ObjectId(),
  subject: "Login issue",
  description: "Unable to login into account",
  department: "Technical Support",
  priority: "high",
  status: "open",
  assignedTo: ObjectId(),
  dueDate: ISODate(),
  resolvedAt: null,
  closedAt: null,
  createdAt: ISODate(),
  updatedAt: ISODate()
}
```

ticket_messages

```
{
  _id: ObjectId(),
  organizationId: ObjectId(),
  ticketId: ObjectId(),
  senderId: ObjectId(),
  senderType: "user",
  message: "Please check your login credentials.",
  attachments: [],
  isInternal: false,
  createdAt: ISODate()
}
```

quotes

```
{
  _id: ObjectId(),
  organizationId: ObjectId(),
  quoteNumber: "QUO-0001",
  customerId: ObjectId(),
  contactId: ObjectId(),
  dealId: ObjectId(),
  items: [{
    productId: ObjectId(),
    name: "CRM Enterprise",
    quantity: 1,
    unitPrice: NumberDecimal("100000.00"),
    taxRate: 18,
    total: NumberDecimal("118000.00")
  }],
  subtotal: NumberDecimal("100000.00"),
  discount: NumberDecimal("0.00"),
  tax: NumberDecimal("18000.00"),
  total: NumberDecimal("118000.00"),
  currency: "PKR",
  status: "sent",
  validUntil: ISODate(),
  createdBy: ObjectId(),
  createdAt: ISODate(),
  updatedAt: ISODate()
}
```

invoices

```
{
  _id: ObjectId(),
  organizationId: ObjectId(),
  invoiceNumber: "INV-0001",
  customerId: ObjectId(),
  dealId: ObjectId(),
  quoteId: ObjectId(),
  items: [{
    productId: ObjectId(),
    name: "CRM Enterprise",
    quantity: 1,
    unitPrice: NumberDecimal("100000.00"),
    taxRate: 18,
    total: NumberDecimal("118000.00")
  }],
  subtotal: NumberDecimal("100000.00"),
```

```
  discount: NumberDecimal("0.00"),
  tax: NumberDecimal("18000.00"),
  total: NumberDecimal("118000.00"),
  paidAmount: NumberDecimal("50000.00"),
  dueAmount: NumberDecimal("68000.00"),
  currency: "PKR",
  issueDate: ISODate(),
  dueDate: ISODate(),
  status: "partially_paid",
  createdAt: ISODate(),
  updatedAt: ISODate()
}
```

payments

```
{
  _id: ObjectId(),
  organizationId: ObjectId(),
  invoiceId: ObjectId(),
  customerId: ObjectId(),
  paymentNumber: "PAY-0001",
  amount: NumberDecimal("50000.00"),
  currency: "PKR",
  method: "bank_transfer",
  transactionReference: "TXN-12345",
  status: "completed",
  paymentDate: ISODate(),
  createdBy: ObjectId(),
  createdAt: ISODate()
}
```

notifications

```
{
  _id: ObjectId(),
  organizationId: ObjectId(),
  userId: ObjectId(),
  title: "New Lead Assigned",
  message: "A new lead has been assigned to you.",
  type: "lead_assigned",
  relatedTo: { type: "lead", id: ObjectId() },
  isRead: false,
  readAt: null,
  createdAt: ISODate()
}
```

files

```
{
  _id: ObjectId(),
  organizationId: ObjectId(),
  fileName: "proposal.pdf",
  originalName: "Client Proposal.pdf",
  mimeType: "application/pdf",
  size: 204800,
  storageDisk: "s3",
  storagePath: "crm/files/proposal.pdf",
  relatedTo: { type: "deal", id: ObjectId() },
  uploadedBy: ObjectId(),
  createdAt: ISODate()
}
```

audit_logs

```
{
  _id: ObjectId(),
  organizationId: ObjectId(),
  userId: ObjectId(),
  action: "updated",
  module: "leads",
  documentId: ObjectId(),
  changes: {
    status: { old: "new", new: "qualified" }
  },
  ipAddress: "192.168.1.1",
  userAgent: "Browser",
  createdAt: ISODate()
}
```

tags

```
{
  _id: ObjectId(),
  organizationId: ObjectId(),
  name: "VIP",
```

```
  color: "#F59E0B",
  createdAt: ISODate()
}
```

custom_fields

```
{
  _id: ObjectId(),
  organizationId: ObjectId(),
  module: "leads",
  label: "Company Annual Revenue",
  name: "annual_revenue",
  type: "number",
  options: [],
  isRequired: false,
  isActive: true,
  order: 1,
  createdAt: ISODate()
}
// Example field values on a lead:
// customFields: { annual_revenue: 5000000, industry_type: "Technology" }
```

settings

```
{
  _id: ObjectId(),
  organizationId: ObjectId(),
  general: {
    dateFormat: "Y-m-d",
    timezone: "Asia/Karachi",
    currency: "PKR"
  },
  numbering: {
    leadPrefix: "LEAD",
    customerPrefix: "CUS",
    dealPrefix: "DEAL",
    invoicePrefix: "INV",
    quotePrefix: "QUO",
    ticketPrefix: "TKT"
  },
  notifications: {
    email: true,
    inApp: true,
    taskReminders: true
  },
  createdAt: ISODate(),
  updatedAt: ISODate()
}
```

4. Additional Production Collections

Collection	Recommended fields / purpose
lead_status_history	Store leadId, organizationId, oldStatus, newStatus, changedBy, reason and createdAt.
deal_stage_history	Store dealId, pipelineId, oldStageId, newStageId, changedBy and createdAt.
counters	Maintain atomic sequences per organization and document type, e.g. LEAD-000123.
email_templates	Store organizationId, name, subject, body, variables and status.
email_campaigns	Store campaign audience, template, schedule, status and creator.
email_logs	Store recipient, provider message ID, delivery status, failure reason and timestamps.
sequences	Store automated steps, delays, conditions and active status for follow-up workflows.
reminders	Store userId, related record, remindAt, status and delivery metadata.
integrations	Store provider name, encrypted credentials/reference, scopes and connection status.
webhooks	Store endpoint, subscribed events, signing secret reference and active status.

5. Relationships and Data Flow

Organization is the tenant root. It owns users, roles, teams, customers, leads, pipelines, products and settings.

Typical sales flow:

Lead → Customer + Contact → Deal → Quote → Invoice → Payment

A lead conversion should be handled as one controlled business operation. Create or match the customer and contact, create the deal if applicable, then mark the lead as converted with references to the created records.

Customer support flow:

Customer → Ticket → Ticket Messages

Activity timeline: Activities, tasks, notes, files and relevant audit events can be queried by the relatedTo type/id pair. Validate that the related record belongs to the same organization.

MongoDB does not enforce relational foreign keys. Validate referenced IDs in the service layer and ensure every referenced document has the same organizationId.

6. Index Strategy

Create indexes based on actual query patterns. The following mongosh script covers the main unique keys and common filters. Run it after cleaning any existing duplicate data.

```
use crm_database;

// Organizations
db.organizations.createIndex({ slug: 1 }, { unique: true });

// Users and roles
db.users.createIndex({ organizationId: 1, email: 1 }, { unique: true });
db.users.createIndex({ organizationId: 1, roleId: 1 });
db.roles.createIndex({ organizationId: 1, slug: 1 }, { unique: true });
db.teams.createIndex({ organizationId: 1, name: 1 });

// Customers and contacts
db.customers.createIndex(
  { organizationId: 1, customerCode: 1 }, { unique: true }
);
db.customers.createIndex({ organizationId: 1, email: 1 });
db.customers.createIndex({ organizationId: 1, assignedTo: 1, status: 1 });
db.contacts.createIndex({ organizationId: 1, customerId: 1 });
db.contacts.createIndex({ organizationId: 1, email: 1 });

// Leads and sources
db.leads.createIndex({ organizationId: 1, leadCode: 1 }, { unique: true });
db.leads.createIndex({ organizationId: 1, status: 1 });
db.leads.createIndex({ organizationId: 1, assignedTo: 1, status: 1 });
db.leads.createIndex({ organizationId: 1, email: 1 });
db.leads.createIndex({ organizationId: 1, createdAt: -1 });
db.lead_sources.createIndex(
  { organizationId: 1, slug: 1 }, { unique: true }
);

// Pipelines and deals
db.pipelines.createIndex({ organizationId: 1, name: 1 });
db.pipeline_stages.createIndex(
  { organizationId: 1, pipelineId: 1, order: 1 }
);
db.deals.createIndex({ organizationId: 1, dealCode: 1 }, { unique: true });
db.deals.createIndex({ organizationId: 1, pipelineId: 1, stageId: 1 });
db.deals.createIndex({ organizationId: 1, assignedTo: 1, status: 1 });
db.deals.createIndex({ organizationId: 1, expectedClosingDate: 1 });

// Products, activities, tasks and notes
db.products.createIndex({ organizationId: 1, sku: 1 }, { unique: true });
db.activities.createIndex({
  organizationId: 1, "relatedTo.type": 1, "relatedTo.id": 1
});
db.activities.createIndex({ organizationId: 1, assignedTo: 1, scheduledAt: 1 });
db.tasks.createIndex({
  organizationId: 1, assignedTo: 1, status: 1, dueDate: 1
});
db.tasks.createIndex({ organizationId: 1, reminderAt: 1 });
db.notes.createIndex({
  organizationId: 1, "relatedTo.type": 1, "relatedTo.id": 1
});

// Support
db.tickets.createIndex(
  { organizationId: 1, ticketNumber: 1 }, { unique: true }
);
db.tickets.createIndex({ organizationId: 1, assignedTo: 1, status: 1 });
```

```
db.ticket_messages.createIndex(
  { organizationId: 1, ticketId: 1, createdAt: 1 }
);

// Quotes, invoices and payments
db.quotes.createIndex(
  { organizationId: 1, quoteNumber: 1 }, { unique: true }
);
db.invoices.createIndex(
  { organizationId: 1, invoiceNumber: 1 }, { unique: true }
);
db.invoices.createIndex({ organizationId: 1, customerId: 1, status: 1 });
db.payments.createIndex({
  organizationId: 1, invoiceId: 1, paymentDate: -1
});

// Notifications, files, tags and configuration
db.notifications.createIndex({
  organizationId: 1, userId: 1, isRead: 1, createdAt: -1
});
db.files.createIndex({
  organizationId: 1, "relatedTo.type": 1, "relatedTo.id": 1
});
db.tags.createIndex(
  { organizationId: 1, name: 1 }, { unique: true }
);
db.custom_fields.createIndex(
  { organizationId: 1, module: 1, name: 1 }, { unique: true }
);
db.audit_logs.createIndex({
  organizationId: 1, userId: 1, createdAt: -1
});
db.settings.createIndex({ organizationId: 1 }, { unique: true });

print("CRM indexes created successfully!");
```

Important: A unique compound index such as { organizationId: 1, email: 1 } also treats missing/null values as index keys. If multiple documents may omit the field, consider a partialFilterExpression or a carefully designed sparse/partial unique index. Confirm index behavior against your MongoDB version and data.

7. Validation, Security and Production Rules

• **Tenant isolation:** Every read, update and delete must be scoped by organizationId. Never trust organizationId supplied by an untrusted client; derive it from the authenticated session/token.

• **Authorization:** Check both role permissions and record-level access (for example, whether a sales agent may view another agent's leads).

• **Passwords:** Store only Argon2id or bcrypt hashes. Never return password or credential fields in API responses.

• **Money:** Use BSON Decimal128 for monetary values; avoid JavaScript floating-point arithmetic for financial totals.

• **Transactions:** Use MongoDB transactions for multi-document operations such as lead conversion and payment posting. Transactions require a replica set or a supported sharded deployment.

• **Soft deletion:** Add deletedAt where recovery/audit requirements apply. Make normal queries exclude deleted records.

• **Validation:** Validate enums, required fields, formats, ranges and references in the application. Add MongoDB collection validators for critical structures.

• **Numbering:** Use an atomic counters collection with findOneAndUpdate and $inc; do not generate invoice/lead numbers by counting documents.

• **Audit:** Record who changed important fields, when, and the before/after values. Avoid logging passwords, tokens or sensitive payment data.

• **Backups:** Configure automated backups, retention and restore testing. Indexes should be based on measured query patterns and explain() output.

• **File storage:** Store file metadata in MongoDB; keep file bytes in object storage or a dedicated file service unless there is a clear GridFS requirement.

• **Pagination:** Use cursor/range pagination for large datasets and cap page sizes.

8. Suggested Backend Project Structure

```
crm/
■■■ src/
■   ■■■ config/
■   ■   ■■■ database.js
■   ■■■ models/
■   ■■■ controllers/
■   ■■■ services/
■   ■■■ repositories/
■   ■■■ middleware/
■   ■■■ routes/
■   ■■■ validators/
■   ■■■ utils/
■   ■■■ app.js
■■■ database/
■   ■■■ seeds/
■   ■■■ indexes/
■■■ tests/
■■■ .env
■■■ package.json
```

Recommended stack: Node.js + Express + MongoDB + Mongoose. Keep business logic in services, database access in repositories, request validation in validators, and authorization/tenant scoping in middleware plus service-level checks.

9. Implementation Checklist

• Create the crm_database database and configure MongoDB connection.

• Create organization registration and tenant provisioning.

• Implement authentication, password hashing, sessions/tokens and account verification.

• Implement roles, permissions and team membership.

• Build customers, contacts and lead management APIs.

• Build pipeline stages, deals and lead conversion workflow.

• Build activities, tasks, notes and reminders.

• Build products, quotations, invoices and payment posting.

• Build ticketing and ticket message history.

• Add notifications, files, audit logs and custom fields.

• Add indexes and schema validators after confirming query patterns.

• Write integration tests for tenant isolation, permissions, financial calculations and transactions.

• Configure backups, monitoring, rate limits and production secrets.

Scope note: This document is a comprehensive database blueprint, not a fully deployed database or a complete application implementation. Collection validators, Mongoose schemas, seed data, API endpoints and tests should be implemented and reviewed before production use.